

Práctica 2: Modificación Estructural y Redimensionamiento de Imágenes BMP

Datos Generales

Estudiante titular: Luis Fernando Tlapalcoyoac Olivares

Unidad de Aprendizaje: Multimedia

Institución: UPIITA - Instituto Politécnico Nacional (IPN)

1. Introducción

En este desarrollo se exploró la arquitectura interna de las imágenes BMP con el propósito de alterar su geometría original. A diferencia de las modificaciones puramente colorimétricas, el redimensionamiento de un mapa de bits exige la manipulación de los metadatos de la cabecera y el reajuste del alineamiento de memoria impuesto por el formato. El objetivo central consistió en incrementar las dimensiones de la imagen insertando datos aleatorios, validando así la correcta sincronización entre la declaración estructural y el bloque de píxeles real.

2. Metodología y Desarrollo

El procedimiento se dividió en operaciones de lectura, alteración lógica y empaquetado. Inicialmente, se leyeron los primeros 54 bytes correspondientes a la cabecera. Utilizando funciones de desempaqueado a nivel de byte con la librería nativa, se extrajeron las dimensiones originales para calcular los bytes de relleno (padding) que alinean las filas a múltiplos de 4 bytes.

Con la matriz de píxeles en memoria, se procedió a expandir la imagen sumando una unidad al ancho y otra al alto. Para ello, se anexó un nuevo píxel con valores BGR generados aleatoriamente al final de cada fila existente. Posteriormente, se añadió una fila inferior completamente nueva, poblada con píxeles también aleatorios, para coincidir con la nueva anchura.

En la fase final, se recalcularon los requisitos de alineación (nuevo padding) y el peso total del archivo actualizado. Estos nuevos valores se inyectaron directamente en los desplazamientos correspondientes de la cabecera (tamaño total, ancho y alto) antes de ensamblar y escribir el archivo resultante en disco.

3. Código Fuente

```
import struct
import random

def redimensionar_bmp_aleatorio(ruta_img_origen, ruta_img_destino):
    # Fase 1: Lectura de cabecera y extracción de metadatos
    with open(ruta_img_origen, "rb") as archivo_origen:
        cabecera_bytes = bytearray(archivo_origen.read(54))

    # Desempaquetado de ancho y alto desde los offsets 0x12 (18) y 0x16
    ancho_orig = struct.unpack_from("<I", cabecera_bytes, 18)[0]
    alto_orig = struct.unpack_from("<I", cabecera_bytes, 22)[0]

    # Cálculo de bytes nulos para el alineamiento múltiplo de 4
    ajuste_padding_orig = (4 - ((ancho_orig * 3) % 4)) % 4

    matriz_pixeles = []
    for _ in range(alto_orig):
        bytes_linea = archivo_origen.read(ancho_orig * 3)
        archivo_origen.read(ajuste_padding_orig) # Omitir padding anterior

        # Agrupación en bloques BGR (3 bytes)
        bloque_bgr = [bytes_linea[i:i+3] for i in range(0,
len(bytes_linea), 3)]
        matriz_pixeles.append(bloque_bgr)

    # Fase 2: Expansión de dimensiones
    ancho_expandido = ancho_orig + 1
    alto_expandido = alto_orig + 1

    # Inserción de una nueva columna (un píxel al final de cada fila actual)
    for fila in matriz_pixeles:
        color_random = bytes([random.randint(0, 255), random.randint(0, 255),
random.randint(0, 255)])
        fila.append(color_random)

    # Inserción de una nueva fila base completa
    linea_extra = [bytes([random.randint(0, 255), random.randint(0, 255),
random.randint(0, 255)]) for _ in range(ancho_expandido)]
    matriz_pixeles.append(linea_extra)

    # Fase 3: Recálculo estructural y actualización de cabecera
    ajuste_padding_nuevo = (4 - ((ancho_expandido * 3) % 4)) % 4
    peso_pixeles_total = ((ancho_expandido * 3) + ajuste_padding_nuevo) *
alto_expandido
    peso_archivo_nuevo = 54 + peso_pixeles_total

    # Modificación in-place de la cabecera con los valores actualizados
    struct.pack_into("<I", cabecera_bytes, 2, peso_archivo_nuevo)
    struct.pack_into("<I", cabecera_bytes, 18, ancho_expandido)
    struct.pack_into("<I", cabecera_bytes, 22, alto_expandido)

    # Fase 4: Escritura del nuevo mapa de bits
```

```
with open(ruta_img_destino, "wb") as archivo_final:
    archivo_final.write(cabecera_bytes)

    secuencia_padding = b'\x00' * ajuste_padding_nuevo
    for fila in matriz_pixeles:
        for pixel in fila:
            archivo_final.write(pixel)
        archivo_final.write(secuencia_padding)

# Ejecución del script
redimensionar_bmp_aleatorio("./images/example001.bmp", "output_modificado.bmp")
```

4. Conclusiones

La alteración geométrica de un mapa de bits trasciende la simple adición de datos, requiriendo una sincronización estricta entre la información declarada en la cabecera y el cuerpo del archivo. La gestión del padding probó ser el factor más crítico, ya que un mal cálculo de este alineamiento corrompe el desplazamiento de las filas, distorsionando la imagen por completo. Esta intervención directa mediante la manipulación de bytes consolidó la comprensión de la estructuración de datos binarios y las especificaciones técnicas del contenedor BMP.